Hospital Management System (HMS)

A Full-Stack Web Application with Role-Based Access Control

Presented by:

Abhigyan Varma-1032232985

Ameya Nangle-1032233045

Course: Database Management System

Department of Electrical and Electronics Engineering

The Problem: Inefficient Legacy Systems

The healthcare industry fundamentally relies on timely access to accurate patient data. However, many small and medium-sized clinics still rely on outdated or paper-based management systems.

Inefficiency & Bottlenecks

Manual record-keeping and search are time-consuming, creating administrative bottlenecks and slowing down patient care.

Data Loss & Security Risk

Paper-based methods are prone to data loss, physical damage, and lack modern encryption standards, posing a critical privacy risk.

Disjointed Data Silos

Information often remains siloed—schedules are separate from notes, and pharmacy stock is disconnected from prescriptions.

Project Aim and Core Objectives

Our primary aim is to design, develop, and implement a secure, scalable, multi-user web application to manage core hospital workflows effectively.

Database Integrity

Design a normalized (3NF) relational database using MySQL to ensure maximum data consistency and integrity.

Secure Authentication

Implement robust, bcrypt-hashed authentication and strict Role-Based Access Control (RBAC).

RESTful Backend

Build a secure and performant RESTful API using Node.js and Express.js to handle business logic.

Dynamic Frontend

Develop a dynamic, component-based React frontend with three tailored dashboards for different user roles.

Limitations of Existing Solutions

While the need for digitalization is clear, existing commercial solutions often present their own set of challenges for smaller practices.

Enterprise EHR Systems

Major Electronic Health Records (EHR) like Cerner or Epic are:

- **Prohibitively Expensive:** High licensing and maintenance costs.
- **Overly Complex:** Designed for large hospitals, offering too many features that overwhelm small clinics.
- **Difficult to Integrate:** Require extensive IT infrastructure and setup time.

> Our project addresses this gap by offering a lightweight, secure, and affordable solution tailored for the 80% of common clinic use cases.

Proposed System Architecture: Role-Based Portals

The proposed system is a three-tier web application centralizing all hospital data into a single, secure MySQL database. Access is strictly managed by user role.

Patient Portal

- Secure Registration/Login
- Schedule & Cancel Appointments
- View Appointment History
- Doctor Rating/Feedback

Doctor Portal

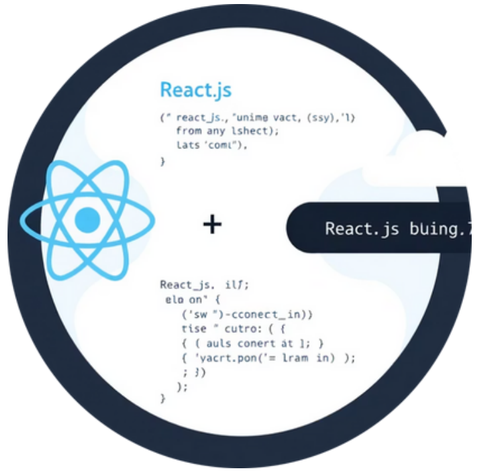
- Personalized Appointment Queue
- View Full Medical History
- Create New Medical Records
- Secure File Uploads (X-rays, Labs)

Receptionist Portal

- Full Patient Directory Access
- Manage Hospital Schedule
- Mark Appointments Complete
- Medicine/Inventory Stock Management

Technology Stack and Tools

The system is built on a modern, robust MERN-like stack, utilizing JavaScript on the frontend and backend, with MySQL for persistent data storage.



Frontend

React.js, Vite, TailwindCSS

Component-based architecture ensures a modular, maintainable, and responsive user interface.

Backend

Node.js, Express.js

High-performance backend for building the secure, stateful RESTful API endpoints.

Database

MySQL

Reliable relational database chosen for data integrity and transactional support.

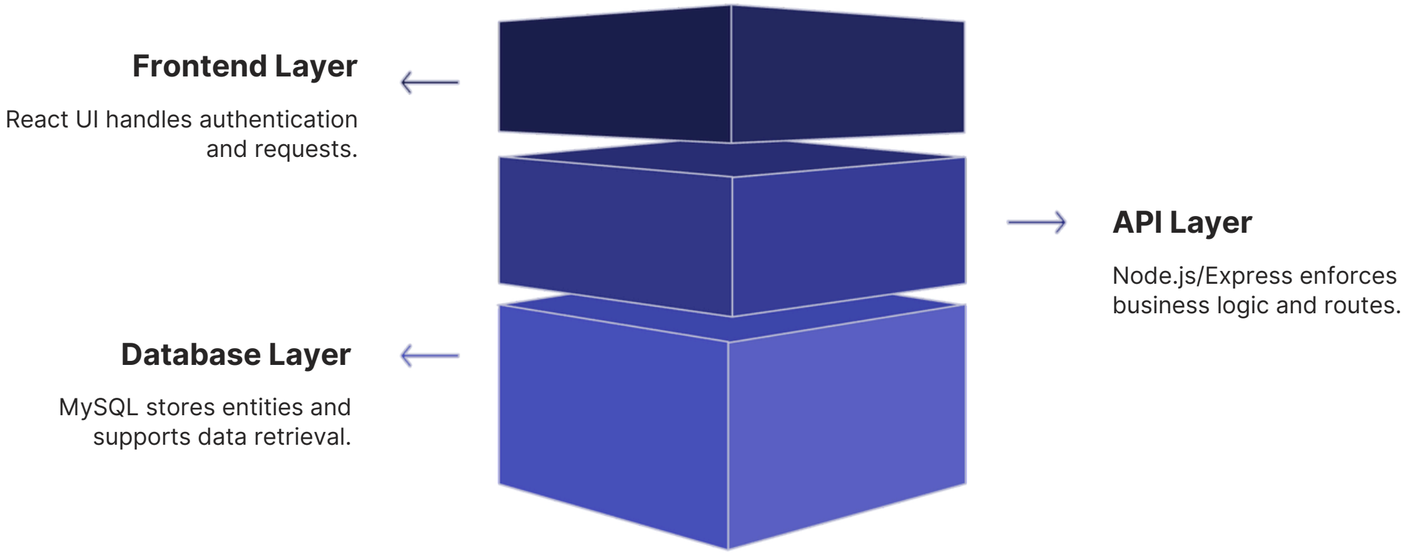
Core Libraries

bcrypt, multer, mysql2, cors

Essential dependencies for hashing passwords, handling file uploads, and database connection.

System Design: Data Flow and Structure

The architecture prioritizes a clear separation of concerns, ensuring that the database remains the single source of truth and the API handles all business logic.



Key Relationships (ER Diagram Focus)

- **Core Entities:** Patient, Doctor, and Appointment tables form the central relational structure.
- **Linkage:** The MedicalRecord table links a Patient to a Doctor and is optionally tied to an Appointment.
- **Administrative:** Medicine and Bill tables support inventory management and financial tracking.

Implementation: Key Workflows

Implementation focused on critical, secure processes essential for daily hospital operations, particularly login and data recording.

Patient Login Workflow

Client (PatientLogin.jsx) sends phone number and password via POST to /api/patients/login.

Server Authentication

Server-side logic compares the submitted password with the stored **bcrypt hash** using bcrypt.compare().

Session Management

Upon successful validation, the server returns the user object and the frontend sets the currentUser state for RBAC.

Doctor File Upload Workflow

1. Doctor uploads a file (e.g., X-ray) using AddRecordForm.jsx to the /api/upload route.
2. **Multer middleware** processes the request, saves the file to the /uploads/ directory on the server.
3. The form submits the new Medical Record, including the generated **file path**, to /api/records.
4. The server INSERTs a new row into the MedicalRecord table with the file's storage path.

Results and Validation (Test Cases)

A series of unit and integration test cases were executed to validate the core system workflows and ensure data integrity across all user portals.

TC01	New Patient Registration: Verify patient is created with a bcrypt-hashed password.	**Pass:** Patient record created; successful auto-login.
TC03	Appointment Scheduling: Ensure new appointment appears in Appointment table and Receptionist dashboard.	**Pass:** Real-time visibility verified.
TC04	Cancel Appointment: Validate that a PATCH request updates the status field to 'Cancelled'.	**Pass:** Status flag updated correctly.
TC06	Doctor File Upload: Confirm file is stored in /uploads/ and the path is saved in the MedicalRecord table.	**Pass:** Storage and database linkage confirmed.
TC07	Stock Management: Verify PATCH request correctly updates the Medicine inventory count.	**Pass:** Integer modification successful via API.

Conclusion and Future Scope

The Hospital Management System successfully demonstrates a secure, full-stack application that fulfills all core objectives, leveraging a normalized MySQL schema, a Node.js API, and role-based React dashboards.

Cloud Deployment

Migrate the database to a managed cloud service (e.g., AWS RDS) and use AWS S3 for scalable file storage (replacing local Multer storage).

Enhanced Staff Security

Apply bcrypt hashing to all staff and doctor passwords, replacing the current placeholder security for these tables.

Billing & Prescriptions Module

Expand API routes and UI components to fully utilize the existing Bill and Prescription tables for complete operational coverage.